

SQL Security Multi-Agent

NL-запрос → безопасный PostgreSQL-SQL с аудитом

GreenData Case 3

Hybrid auditor

Reflection-loop

user: Покажи топ договоров по сумме

```
SELECT id, credit_amount  
FROM credit_contract  
ORDER BY credit_amount DESC  
LIMIT 10;
```

audit: approved · risk 0.0
artifact: SQL + audit_log

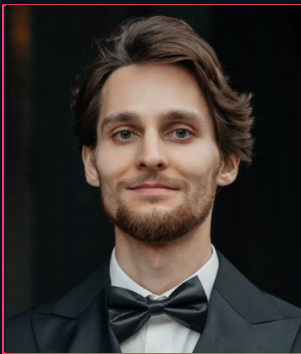
Команда проекта

Четыре зоны ответственности — один сквозной security pipeline



Серафим

Schema layer · DDL parser ·
frontend



Григорий

Security auditor · rules · sensitive
detector



Алена

LLM SQL generator · prompt ·
reflection-loop



Михаил

Orchestrator · dataset · infra ·
observability



Задача: NL→SQL, но под контролем безопасности

Система возвращает SQL + audit_log; execution на production остаётся на стороне заказчика

Пользователь:

«Покажи все данные клиентов»

Наивный LLM:

```
SELECT * FROM clients;
```

Риск:

sensitive-поля · широкая выгрузка · нет LIMIT

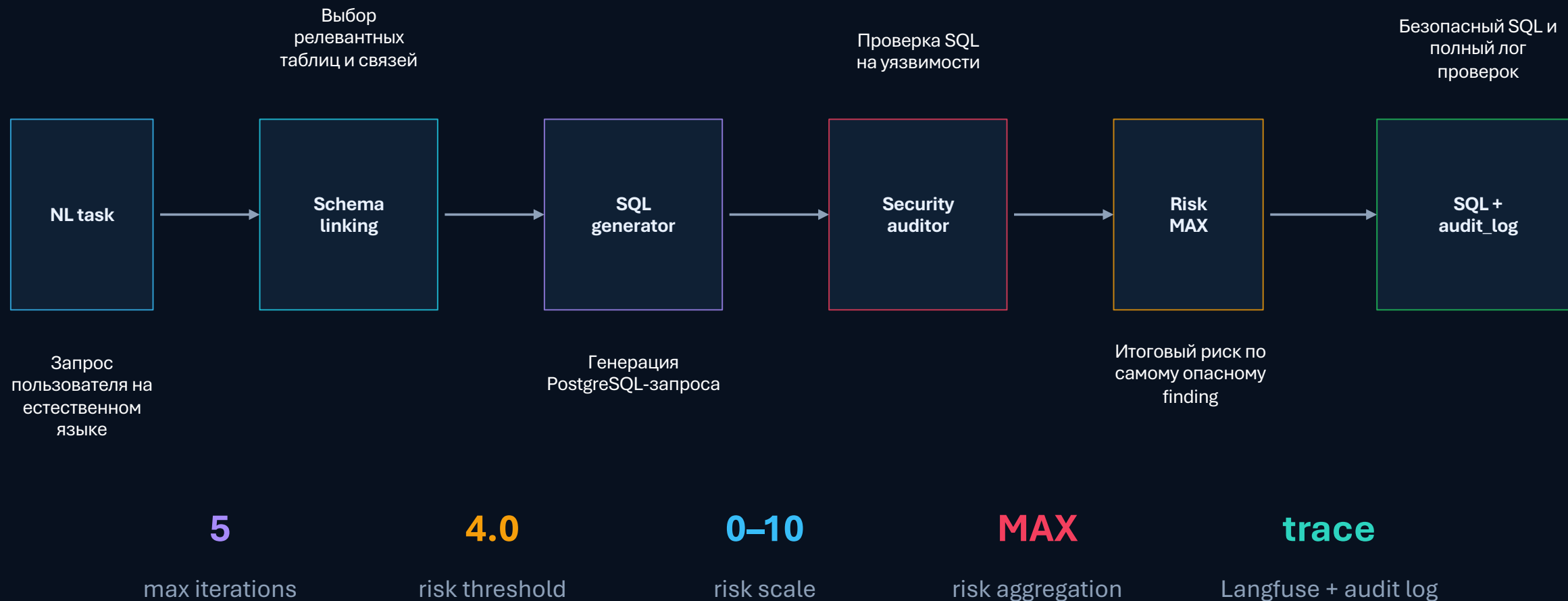
Что фиксирует система

- одобренный SQL или rejected-результат
- полный audit_log по итерациям
- risk_score по шкале 0–10
- объяснение findings и рекомендации

По ТЗ: не исполняем SQL на проде заказчика

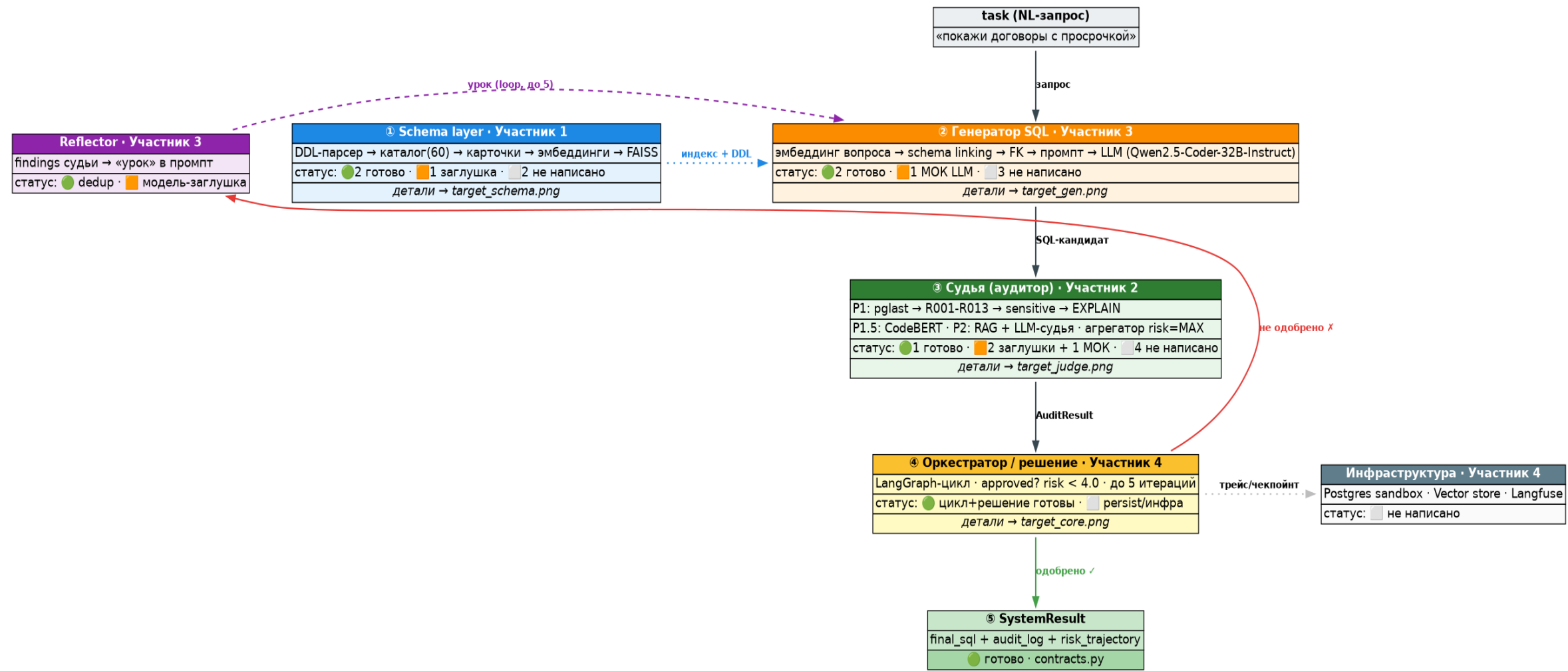
Архитектура решения

Гибрид: LLM-генератор + аудитор + rules + RAG + observability



Архитектура решения (схема)

Гибрид: LLM-генератор + аудитор + rules + RAG + observability



Сильные стороны проекта

Не просто генерация SQL, а управляемый контур безопасности

Schema-grounded

Schema-grounded

Модель получает релевантный фрагмент схемы, а не всю БД целиком.

Hybrid auditor

Hybrid auditor

Правила, sensitive-детектор, LLM-судья и RAG сводятся через MAX risk.

Reflection-loop

Reflection-loop

Finding превращается в Lesson и исправляет следующую генерацию.

Explainability

Explainability

Каждая итерация сохраняется в audit_log: SQL, risk, findings, verdict.

Observability

Observability

Prometheus/Grafana показывают метрики, Langfuse хранит traces и scores.

Reproducibility

Reproducibility

Docker Compose + setup.sh + make check поднимают стек быстро.

Schema layer: из DDL в релевантный контекст

Серафим: parser · catalog · cards · embeddings · FAISS · frontend

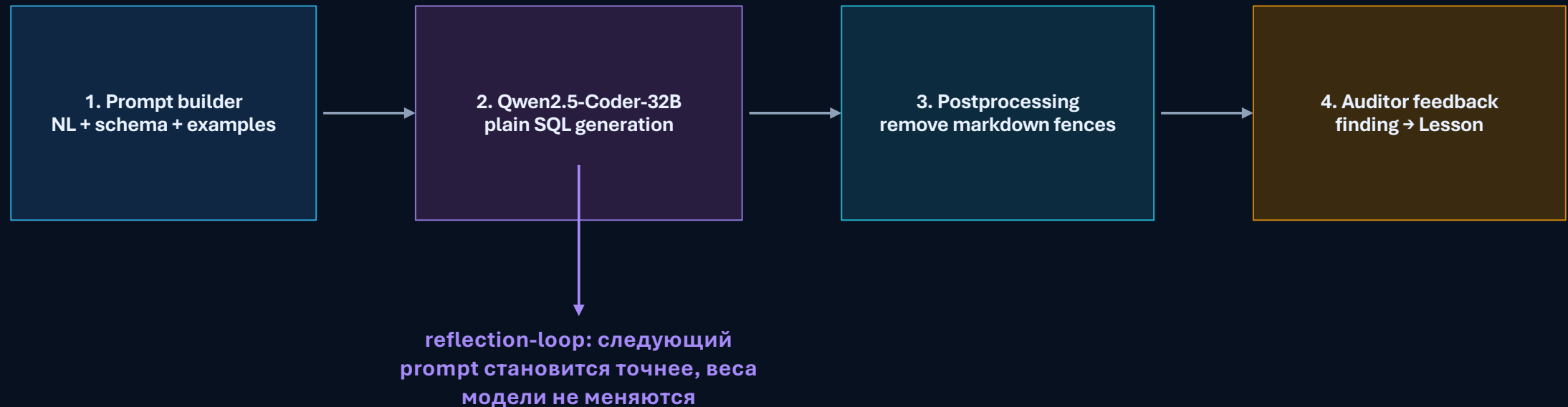


Итог: генератор видит точный релевантный фрагмент схемы; аудитор может ловить out-of-schema SQL.

LLM-генератор: prompt как контракт

Алена: SQL generation · few-shot · postprocessing · reflection-loop

На вход генератор получает не только NL-запрос, но и подготовленный контекст: schema block, few-shot примеры и lessons от аудитора.



System rules: no SELECT * · use LIMIT · only schema names · avoid sensitive output

Аудитор: безопасность через несколько слоёв

Григорий: 9 классов рисков · sensitive-detector · LLM-judge · MAX aggregation

Аудит

Уязвимости

Grafana

Langfuse

Админка

КАТАЛОГ УЯЗВИМОСТЕЙ (9 КЛАССОВ ИЗ ТЗ)

Левая полоса карточки — уровень риска: зелёная INFO · оранжевая PII / средний · красная CRITICAL / DESTRUCTIVE. Кнопка «Вставить пример в аудит» подставит NL-вариант в поле на вкладке «Аудит», а запускать цикл (или сначала отредактировать) — уже вам.

CRITICAL

risk 9.0 / 10

SQL_INJ_CLASSIC

Классическая SQL-инъекция

Внедрение SQL через незаэкранированный ввод (комментарии, тавтологии 'OR 1=1').

ПРИМЕР SQL

SELECT * FROM users WHERE name = 'admin' OR '1'='

Почему опасно: Условие WHERE становится всегда истинным → утечка всех строк таблицы.

Вставить NL в поле

Сразу аудит этого SQL

CRITICAL

risk 8.0 / 10

SQL_INJ_UNION

UNION-инъекция

Подклеивает чужой SELECT через UNION — выгребает данные из других таблиц.

ПРИМЕР SQL

SELECT id FROM products WHERE id=1 UNION SELECT p

Почему опасно: Атакующий читает таблицы, к которым нет прямого доступа в эндпоинте.

Вставить NL в поле

Сразу аудит этого SQL

CRITICAL

risk 9.0 / 10

SQL_INJ_TIME

Time-based blind

Использует pg_sleep/CASE — задержка ответа выдаёт значение бита данных.

ПРИМЕР SQL

SELECT 1 FROM users WHERE id=1 AND (CASE WHEN sub

Почему опасно: Слепая эксфильтрация бита за битом — медленно, но без вывода в ответе.

Вставить NL в поле

Сразу аудит этого SQL

DESTRUCTIVE

risk 7.0 / 10

DML_NO_WHERE

UPDATE/DELETE без WHERE

Массовая модификация: WHERE забыт, удалён или экранирован комментарием.

DESTRUCTIVE

risk 10.0 / 10

PRIV_ESCALATE

Повышение прав / DDL

DROP/CREATE/ALTER, GRANT, ALTER USER — разрушение схемы или эскалация привилегий.

CRITICAL

risk 8.0 / 10

PLPGSQL_UNSAFE

Динамический EXECUTE в функции

EXECUTE-конкатенация пользовательского input или SECURITY DEFINER без проверок.

Hybrid judge

- Phase 1: deterministic rules / AST
- Sensitive: PII, cards, passwords, financial fields
- Phase 1.5: CodeBERT malicious score
- Phase 2: LLM-judge + CWE/CAPEC/OWASP RAG
- risk = MAX(finding.risk_score)

Security stance: false positive дешевле, чем false negative — опасный SQL не должен пройти approve.

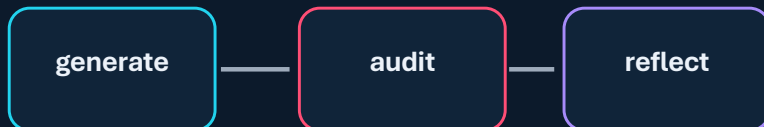
SQL Security · Multi-Agent

09

Оркестратор и инфраструктура

Михаил: управляемый цикл generate → audit → reflect · sandbox DB · metrics · traces · setup.sh

Контролируемый цикл



Если approved → результат

Если risk ≥ 4.0 → lesson в следующий prompt

- максимум 5 итераций
- сохраняем SQL, findings, risk, exit_reason
- ошибки generator/auditor трактуются как rejected

Sandbox DB

Postgres + DDL
заказчика + faker-
данные

7

docker services

Metrics

Prometheus: runs,
latency, iterations,
risk

500

faker rows / table

Tracing

Langfuse: trace,
scores, feedback

setup.sh

idempotent start

Воспроизводимый запуск

setup.sh → docker compose → healthcheck → UI / Grafana / Langfuse

Продуктовый слой: UI, live-stream и audit-log

Пользователь видит не только SQL, но и путь проверки

UI демонстрирует основной сценарий: ввод NL-задачи, live-stream итераций, финальный SQL, vulnerabilities, audit_log и feedback.

Ключевые функции

- FastAPI SSR
- SSE thinking-stream
- clarify-questions
- demo_db execution only
- feedback → jsonl + Langfuse score

SQL Security · Multi-Agent

NL→SQL с аудитом. Цикл «генератор → судья → reflection-фикс». Артефакт: SQL + audit log (без исполнения на проде). Кнопка «Выполнить» на этой странице — ТОЛЬКО для демо на seed-БД.

АудитGrafanaLangfuseАдминка

NL-ЗАДАЧА (НОВЫЙ ДИАЛОГ)

первые 100 записей компаний

НачатьLiveСбросить диалогагрегатjoin-top-NDELETE (clarify)«активный» (clarify)провокация: SELECT *провокация: pg_catalog

первые 100 записей компаний

thinking...

Итерация 1 — 0.1s

генератор пишет SQL... 0.1s

✓ SQL готов

SELECT id, name, short_name, inn, type_id, org_id, status, create_date, user_id, attr_email, attr_web_site, is_system, main FROM sys_company ORDER BY 1

3.4s

аудитор проверяет... 3.4s

rejected · риск 7.0 · DIRECT_SENSITIVE(7.0), DIRECT_SENSITIVE(5.0) 7.2s

reflection пишет урок... 7.2s

✓ урок: [DIRECT_SENSITIVE] Чувствительные поля маскируй или агрегируй, не выбирай сырыми. 7.2s

Итерация 2 — 7.2s

генератор пишет SQL (учтены уроки: 1)... 7.2s

✓ SQL готов

SELECT id, name, short_name, LEFT(inn, 4) || '***' AS inn, type_id, org_id, status, create_date, user_id, LEFT(attr_email,

Продуктовый слой: UI, live-stream и audit-log

Пользователь видит не только SQL, но и путь проверки

UI демонстрирует основной сценарий: ввод NL-задачи, live-stream итераций, финальный SQL, vulnerabilities, audit_log и feedback.

- Ключевые функции
- FastAPI SSR
 - SSE thinking-stream
 - clarify-questions
 - demo_db execution only
 - feedback → jsonl + Langfuse score

✓ урок: [DIRECT_SENSITIVE] Чувствительные поля маскируй или агрегируй, не выбирай сырыми. 7.2s

— Итерация 2 — 7.2s

генератор пишет SQL (учтены уроки: 1)...

✓ SQL готов

SELECT id, name, short_name, LEFT(inn, 4) || '***' AS inn, type_id, org_id, status, create_date, user_id, LEFT(attr_email, 4) || '***' AS attr_email, i

18.1s

аудитор проверяет...

approved · риск 0.0

18.1s

SQL готов · ✓ approved · итераций 2 · риск 7.0 → 0.0

SQL готов · ✓ approved · итераций 2 · риск 7.0 → 0.0

ФИНАЛЬНЫЙ SQL

SELECT id, name, short_name, LEFT(inn, 4) || '***' AS inn, type_id, org_id, status, create_date, user_id, LEFT(attr_email, 4) || '***' AS attr_email, i

FROM sys_company

ORDER BY id ASC

LIMIT 100;

Выполнить на demo_db →

Открыть trace 195adc4b →

УЯЗВИМОСТИ (ПОСЛЕДНЯЯ ИТЕРАЦИЯ)

уязвимостей не найдено

AUDIT LOG

ORDER BY id ASC

LIMIT 100;

Риск: 7.0 Одобрено: False

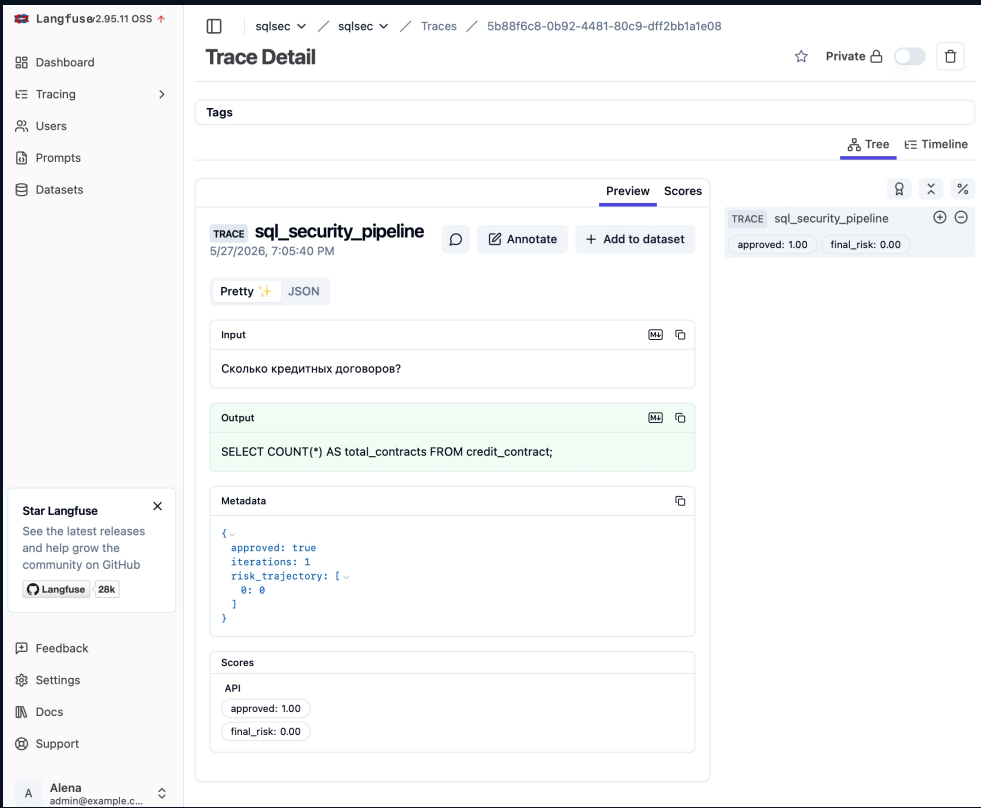
△ DIRECT_SENSITIVE (7.0): Чувствительная колонка 'inn' (national_id) – Выгрузка PII/коммерческой тайны (паспорт, карта, суммы) нарушает 152-ФЗ/PCI D

↳ фикс: Удалите поля attr_email и inn из запроса, если они не требуются для бизнес-логики, или примените соответствующие меры защиты данных, та

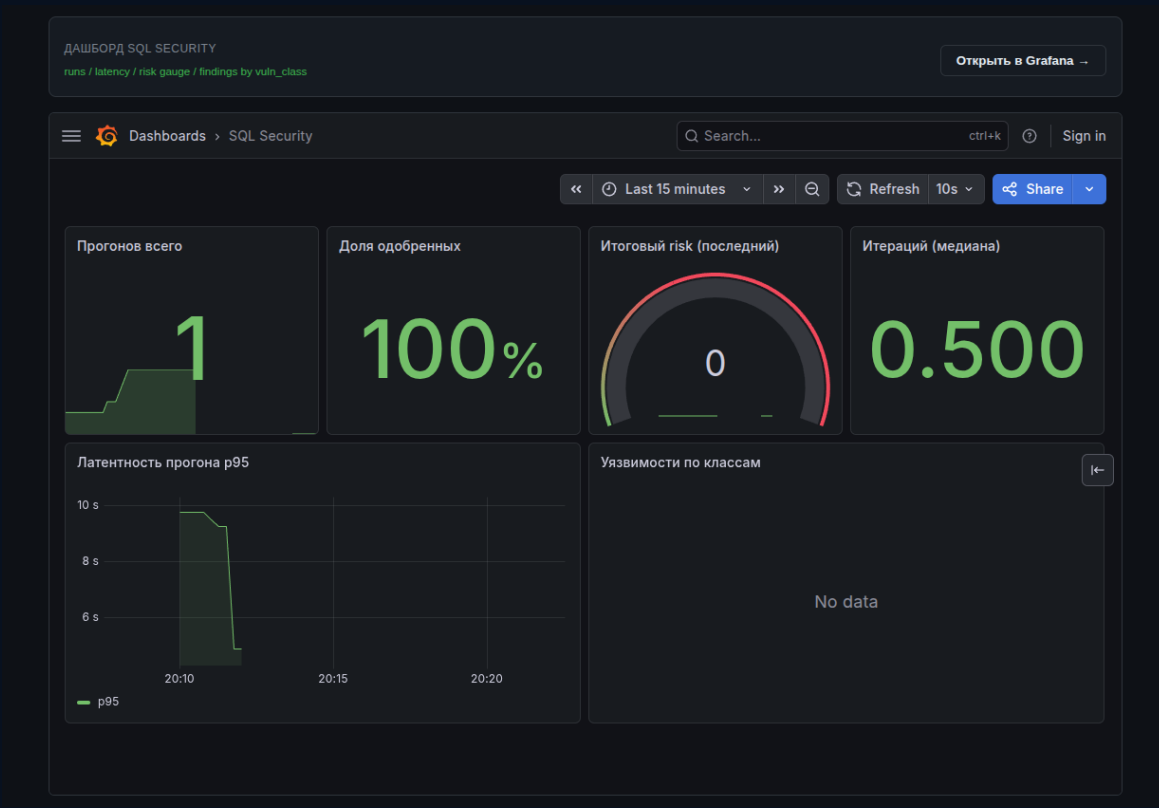
△ DIRECT_SENSITIVE (5.0): Чувствительная колонка 'attr_email' (contact) – Выгрузка PII/коммерческой тайны (паспорт, карта, суммы) нарушает 152-ФЗ/PC

Observability: контроль качества в реальном времени

Trace в Langfuse + метрики в Grafana



Langfuse: prompt · output SQL · iterations · approved · final_risk



Grafana: runs_total · latency p95 · risk gauge · findings by class

Результаты и проверяемость

Воспроизводимый запуск, smoke-test, security-сценарии и наблюдаемость каждого прогона

1 cmd

./setup.sh

идемпотентный старт

make check

smoke-test

контейнеры · endpoints · /audit · metrics

9 classes

security risks

уязвимости из T3

trace

Langfuse

scores · feedback · audit trail

Проверенные демо-сценарии

- ✓ простой NL-запрос → approved SQL + audit_log
- ✓ широкий запрос → clarifying question вместо SELECT *
- ✓ pg_catalog → rejected, risk 4.0, SCHEMA_INTROSPECT
- ✓ feedback → data/feedback.jsonl + Langfuse score

Что видно после запуска

Prometheus: runs, latency, iterations, final risk
Grafana: SQL Security dashboard
Langfuse: trace, prompt/output, scores, feedback
audit_log: SQL и findings по каждой итерации

Результат можно не только показать в UI, но и воспроизвести, проверить smoke-test'ом и отследить в observability-стеке.

setup.sh → make check → UI demo → Grafana metrics → Langfuse trace

Безопасный NL→SQL: генерация, аудит, исправление и наблюдаемость

Главная ценность → это не «LLM пишет SQL», а контролируемый security layer между пользователем и БД

Работа уже закрывает

- NL→PostgreSQL по схеме заказчика
- audit_log и risk score по итерациям
- clarify / reject для опасных запросов

Сильные стороны

- schema-grounded context снижает галлюцинации
- hybrid audit через rules + sensitive + LLM/RAG
- reflection исправляет ошибки без дообучения

Задел развития

- AST-parser для PostgreSQL вместо regex
- расширение правил, CodeBERT и RAG
- vLLM / k8s / per-user memory